Assignment 02: Xv6 System Calls (syscall)

CSci 530: Operating Systems

Objectives

In the last assignment you used some system calls (like `pause()`, `fork()` and `exec()`) to write a few utilities. In this assignment you will add a new system call to xv6, which will help you understand how they work and will expose you to some of the internals of the xv6 kernel. You will need to add more system calls in later assignments.

-

Description

Have a look at the Getting Started instructions for information on how to set up your computer to work on and run these assignments.

Note: Remember that assignments are submitted to GitHub classrooms by creating a commit and pushing it back to your assignment repository. You are required to make a commit for at least each task as you complete it (though you can make more than one commit for a task if needed). In general, from the command line, you can create and push a commit by doing something like:

```
$ git commit -am "My solution for Assignment 02 Task 02, sandbox a command"
$ git push
```

Assignment Tasks

Task 01: Using `gdb` debugger (easy)

In many cases, print statements will be sufficient to debug your kernel, but sometimes it is useful to single step through code or get a stack backtrace. The `gdb` debugger can help.

To help you become familiar with `gdb`, run `make CPUS=1 qemu-gdb` and then fire up `gdb` in another window (see the `gdb` materials on the Additional Resources page from our class).

Note: RISC-V qemu is capable of emulating a multi-core cpu, and by default starts emulating with 2 cpus. However, when debugging, it is often confusing if there are additional cpu cores running, so it is recommended to always compile and run gdb qemu emulator using a single cpu emulated core.

Running `make CPUS=1 qemu-gdb` should look similar to the following, assuming your environment is set up with the tools and code to build xv6:

```
(base) student@964702a18101:/workspaces/assg02-xv6-solution$ make CPUS=1 qemu-gdb
riscv64-linux-gnu-gcc -g -c -o kernel/entry.o kernel/entry.S
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -DSOL_SYSCALL
    -DLAB_SYSCALL -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib
    -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset
    -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero
    -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc
    -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf
    -fno-builtin-vprintf -I. -fno-stac
...
```

```
balloc: first 974 blocks have been allocated
balloc: write bitmap block at sector 46
*** Now run 'gdb' in another window.
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1
    -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0
    -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0 -S -gdb tcp::26000
```

This command should compile the xv6 system and run it in the qemu emulator in debug mode. This runs `gdb` as a server, in the output it is running on the local host on port `26000`, though a different port may be chosen. As mentioned, open up a second terminal window in your DevContainer. From the top-level directory, there should be a `.gdbinit` file that has the information needed to connect to the running `gdb` server:

```
(base) student@964702a18101:/workspaces/assg02-xv6-solution$ cat .gdbinit
set confirm off
set architecture riscv:rv64
target remote 127.0.0.1:26000
symbol-file kernel/kernel
set disassemble-next-line auto
set riscv use-compressed-breakpoints yes
```

Unfortunately current versions of `gdb` will not automatically load this `.gdbinit` file. You need to perform the following command first, to specify it is safe to load this file:

```
(base) student@964702a18101:/workspaces/assg02-xv6-solution$
    mkdir -p ~/.config/gdb; echo "set auto-load safe-path /" >> ~/.config/gdb/gdbinit
```

This basically creates a global `gdb` configuration file that specifies it is safe to load local `.gdbinit` files.

Now you should be able to start `gdb`, it will use the information in `.gdbinit` to connect to your running qemu xv6 emulation. In the gdb window type the following:

```
(base) student@964702a18101:/workspaces/assg02-xv6-solution$ gdb-multiarch -tui
GNU gdb (Debian 16.3-1) 16.3
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
...
(gdb) b syscall
Breakpoint 1 at 0x80001cca: file kernel/syscall.c, line 133.
(gdb) c
Continuing.

Breakpoint 1, syscall () at kernel/syscall.c:133
133     {
(gdb) backtrace
```

The `-tui` flag start gdb in a textual window layout, so that you can see the source code of your current break point in the window above the `gdb` command line. The `b` command sets a breakpoint on the `syscall()` function, and `c` causes the debugger to continue executing until a breakpoint is reached. The `backtrace` command we showed as the final example prints a stack backtrace.

Answer the following questions in 'answers-syscall.txt:

Question: Looking at the backtrace output, which function called syscall?

Type `n` (next step) a few times to step past `struct proc *p = myproc();` Once past this statement, type `p /x *p` which prints the current process's `proc struct` (see `kernel/proc.h`) in hex.

Question: What is the value of `p->trapframe->a7` and what does that value represent? Hint: look at `user/init.c`, the first user program `xv6` starts, and its compiled assembly `user/init.asm`.

The processor is running in supervisor mode when in `syscall()`, and you can print privileged registers such as `sstatus` (see the RISC-V privileged instructions for a description):

```
(gdb) p /x $sstatus
```

Question: What was the previous mode the CPU was in?

The xv6 kernel code contains consistency checks whose failure causes the kernel to panic. You may find that your kernel modifications cause panics. For example, replace the statement

```
num = p->trapframe->a7;
```

with

```
num = * (int *) 0;
```

at the beginning of `syscall()`, run `make qemu`, and you will see something similar to

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
scause=0xd sepc=0x80001cda stval=0x0
panic: kerneltrap
```

Quit out of qemu.

To track down the source of a kernel page-fault panic, search for the `sepc` value printed for the panic you just saw in the `kernel/kernel.asm` which contains the assembly for the compiled kernel (Hint: `sepc` is a reference to the program counter pc where the panic occurred).

Question: Write down the assembly instruction the kernel is panicking at. Which register corresponds to the variable `num`?

To inspect the state of the processor and the kernel at the faulting instruction, fire up `gdb` and set a breakpoint at the faulting `epc`, like this:

```
(gdb) b *0x80001cda
Breakpoint 1 at 0x80001cda: file kernel/syscall.c, line 125.
(gdb) c
Continuing.

Breakpoint 1, syscall () at kernel/syscall.c:125
```

Confirm that the faulting instruction is the same one that you modified before to cause the panic.

Question: Why does the kernel crash? Hint: look at figure 3-3 in the text. Is address 0 mapped in the kernel address space? Is that confirmed by the value in `scause` above? (See description of `scause` in RISC-V privileged instructions).

Note that `scause` was printed by the kernel panic above, but often you need to look at additional info to track down the problem that caused the panic. For example, to find out which user process was running when the kernel panicked, you can print the process's name:

```
(gdb) p p->name
```

Question: What is the name of the process that was running when the kernel panicked? What is the process id (`pid`)?

You may want to revisit Using the GNU Debugger as needed. The Additional Resources page for the class also has debugging tips.

Task 02: Sandbox a command (moderate)

In this task you will "sandbox" a process to restrict the system calls it can make. For example, one might sandbox a process to disallow opening files. You'll create a new `interpose()` system call that will specify which system calls the kernel should reject from the calling process. `interpose()` should take two arguments: an integer mask and a path. The mask's bits specify which system calls to reject. The second argument you will use in the next task below, and

for now it will always be "-". For example, in order for a process to prevent itself from using `open()` system calls, it should call `interpose(1 << SYS_open, "-")`, where `SYS_open` is a syscall number from `kernel/syscall.h`. Your implementation should cause the mask to be inherited by children of `fork()`, so that children inherit the paren'ts restrictions.

You've been provided with a `user/sandbox.c` user program that `forks()`, calls `interpose()` in the child, and then execs a program in the child. When you're done implementing `interpose()`, you would use it like this from your shell:

```
$ sandbox 32768 - cat README
cat: cannot open README
```

In this example, `cat README` is the command being sandboxed. 32768 is the mask of system calls to reject, in this example it is `1 << SYS_OPEN`. If your solutions is correct, you should see an error message "cat: cannot open README" as shown.

Your solution gets full points if you pass the tests in `grade-lab-syscall sandbox_mask`

```
$ ./grade-lab-syscall sandbox_mask
```

Some hints:

- Start by adding `$U/_sandbox` to `UPROGS` in your `Makefile`
  - Also don't forget to undo the kernel panic you introduced in task 01 if you haven't already
- Run `make qemu` and you will see that the compiler cannot compile `user/sandbox.c`, because the user-space stubs for `interpose()` system call don't exist yet. Add a prototype for `interpose()` to `user/user.h`, a stub to `user/usys.pl`, and a syscall number to `kernel/syscall.h`. The Makefile invokes the perl script `user/usys.pl`, which produces `user/usys.S`, the actual system call stubs, which use the RISC-V `ecall` instruction to transition to the kernel. Once you fix the compilation issues, run `sandbox 32768 - cat README` in the xv6 shell. It will fail because you haven't implemented the system call in the kernel yet.
- Add a `sys_interpose()` function in `kernel/sysproc.c` that implements the new system call by recording the mask argument in a new field in the `proc` structure (see `kernel/proc.h`). The functions to retrieve system call arguments from user space are in `kernel/syscall.c`, and you can see examples of their use in `kernel/sysproc.c`. Add your new `sys_interpose()` to the `syscalls` array in `kernel/syscall.c`.
- Modify `kfork()` (see `kernel/proc.c`) to copy the mask from the parent to the child process.
- When modifying `kfork()` you may notice it calls `allocproc()`, which can reuse an old process structure from the process table. It would be a good idea to initialize your interpose mask here when a new process is allocated.
- Modify the `syscall()` function in `kernel/syscall.c` to check if the system call must be rejected.

Don't forget to test your solution and to create and push a commit of your working solution for the task once completed.

Task 03: Sandbox with allowed pathnames (easy)

In this task you will extend the sandbox to handle the second command line argument, a specification of a pathname. This extension is only applicable to the `open()` and `exec()` system calls. The second argument of `sys_interpose()` is the pathname allowed. If `open()` or `exec()` is masked but the pathname matches the allowed pathname, then these system calls should be allowed.

When you're done, you should see output like this:

```
$ sandbox 32768 README grep xv6 README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6).  xv6 loosely follows the structure and style of v6,
xv6 is inspired by John Lions's Commentary on UNIX 6th Edition (Peer
(kaashoek,rtm@mit.edu).  The main purpose of xv6 is as a teaching
$ sandbox 32768 x grep xv6 README
grep: cannot open README
```

In this example, the sandbox allows `open()` calls that access `README`, but not any other file (e.g., `x`).

Your solution is correct if it passes all of the sandbox tests in `make grade`.

```
(base) student@964702a18101:/workspaces/assg02-xv6-solution$ make grade
make clean
...
== Test sandbox_mask ==
$ make qemu-gdb
sandbox_mask: OK (2.7s)
== Test sandbox_fork ==
$ make qemu-gdb
sandbox_fork: OK (0.2s)
== Test sandbox_path ==
$ make qemu-gdb
sandbox_path: OK (0.9s)
== Test sandbox_most ==
$ make qemu-gdb
sandbox_most: OK (0.8s)
== Test sandbox_minus ==
$ make qemu-gdb
sandbox_minus: OK (1.1s)

...
```

Some hints:

- Modify `sys_interpose()` to remember the allowed pathname. `argstr()` will be handy to retrieve the pathname. You can declare a buffer of size `MAXPATH` in the `proc` struct.
- You may need `strncmp()` which is one of the standard "C string" library functions implemented for xv6 in `kernel/string.c` declared in `kernel/defs.h`.
- If `open()` or `exec()` are masked, check if the pathname matches the allowed pathname. If so allow the execution of those system calls.

Task 04: Attack xv6 (moderate)

The xv6 kernel isolates user programs from each other and isolates the kernel from user programs. As you saw in the above assignments, an application cannot directly call a function in the kernel or in another user program; instead, interactions occur only through system calls. However, if there is a bug in the kernel's implementation of a system call, an attacker may be able to exploit that bug to break the isolation boundaries. To get a sense for how bugs can be exploited, we have introduced a bug into xv6 and your goal is to exploit that bug to steal a secret from another process.

The bug is that the call to `memset(mem, 0, sz)` in `uvmalloc()` in `kernel/vm.c` to clear a newly-allocated page is omitted when compiling this lab. Similarly, when compiling `kernel/kalloc.c` for this lab the two lines that use memset to put garbage into free pages are omitted. The net effect of omitting these 3 lines (all marked by `ifndef LAB_SYSCALL`) is that newly allocated memory retains the contents from its previous use. Thus an application that calls `sbrk()` to allocate memory may receive pages that have data in them from previous uses. Despite the 3 deleted lines, xv6 mostly works correctly; it even passes most of usertests.

For this task, `user/secret.c` writes a secret string in its memory and then exits (which frees its memory). Your goal is to add a few lines of code to `user/attack.c` to find the secret that a previous execution of `secret.c` wrote to memory, and to print the secret on a line by itself.

Your `attack.c` must work with unmodified xv6 and unmodified `secret.c`. You can change anything to help you experiment and debug, but must revert those changes before final testing and submitting.

The secret program takes the `secret` as an argument. You can test your `attack` program by running `secret` with some argument, then running `attack` and seeing whether attack prints exactly the argument passed to `secret`. Here's a successful run:

```
$ secret xyzzy
$ attack
```

```
xyzzy
$
```

Depending on exactly how you implement your attack, you may need to run attack a second time in order for it to find the secret. The grader runs attack twice, just in case, and is satisfied if either produces the secret.

Some hints:

- Even though `secret` puts something at the start (index 0) of the array it allocates, this array is not necessarily going to end up at the byte 0 of the page it gets loaded into.

From outside xv6, you can use `./grade-lab-syscall attack`, or `make grade`, to see if your attack passes our tests. The secret strings that the tests generate are guaranteed to contain only digits and upper and lower case letters.

As with this example, bugs that do not directly affect correctness can sometimes be exploited to break security. Careful programming and extensive testing can reduce the number of bugs but can't guarantee their absence. xv6 has had bugs in the past, and probably has yet more undiscovered errors. Real kernels, which have much more code than xv6, have a long history of such bugs. For example, see the public Linux vulnerabilities and how to report vulnerabilities.

Submit the Assignment

Time spent

Create a new file, `time.txt`, and put in a single integer, the number of hours you spent on the lab. Make sure you add and commit a final commit with this file as it is part of the autograder score.

Answers

This lab had questions asked in Task 01. Make sure you have a file named `answer-syscall.txt` with the answer for all 6 questions in this file added and pushed as a commit. This file is checked as part of the final autograder score as well.

Assignment submissions are handled by the GitHub classroom autograder. You should create at least 1 commit for each task as you do it, and push it to your GitHub classroom. You can check that the autograder tests are passing in GitHub classroom by looking at your submitted releases. The tests run in the GitHub classroom are the same ones that you can run locally by using `make grade` or `./grade-lab-util test`.

- Please run `make grade` to ensure that your code passes all of the tests. The GitHub classroom autograder will use the same grading program to assign you an autograder evaluation score for your work.

Optional Challenge Exercises

- Find a bug in xv6 that allows an adversary to break process isolation or crash the kernel and let us know. (Side channels such as Meltdown are out of scope, although we may cover them in lecture.)